RingGSW based Universally Composable Oblivious Transfer

Group No. 22

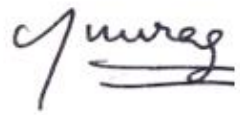
Guide: Dr. Kunwar Singh

Members:

- Anurag Goyal – 106119014
- Manmohan Prajapat – 106119072
- Sobhagya Singh Dewal - 106119117



Dr. Kunwar Singh



Anurag Goyal



Manmohan Prajapat



Sobhagya Singh Dewal

Introduction

We have constructed a secure method for transferring a message without knowing which one the receiver wants. The receiver also can't know the other messages. This is called 1-out-of-n Oblivious Transfer (OT) protocol.

We use a fully homomorphic encryption (FHE) scheme. It allows us to perform operations on ciphertexts in an untrusted environment to achieve the same results on the resulting plaintext.

The protocol is universally composable. Its post-quantum security comes from the RLWE hard problem. We achieved this by converting Craig Gentry's FHE scheme (GSW) to its ring variant (RGSW).

Problem Statement and Objectives

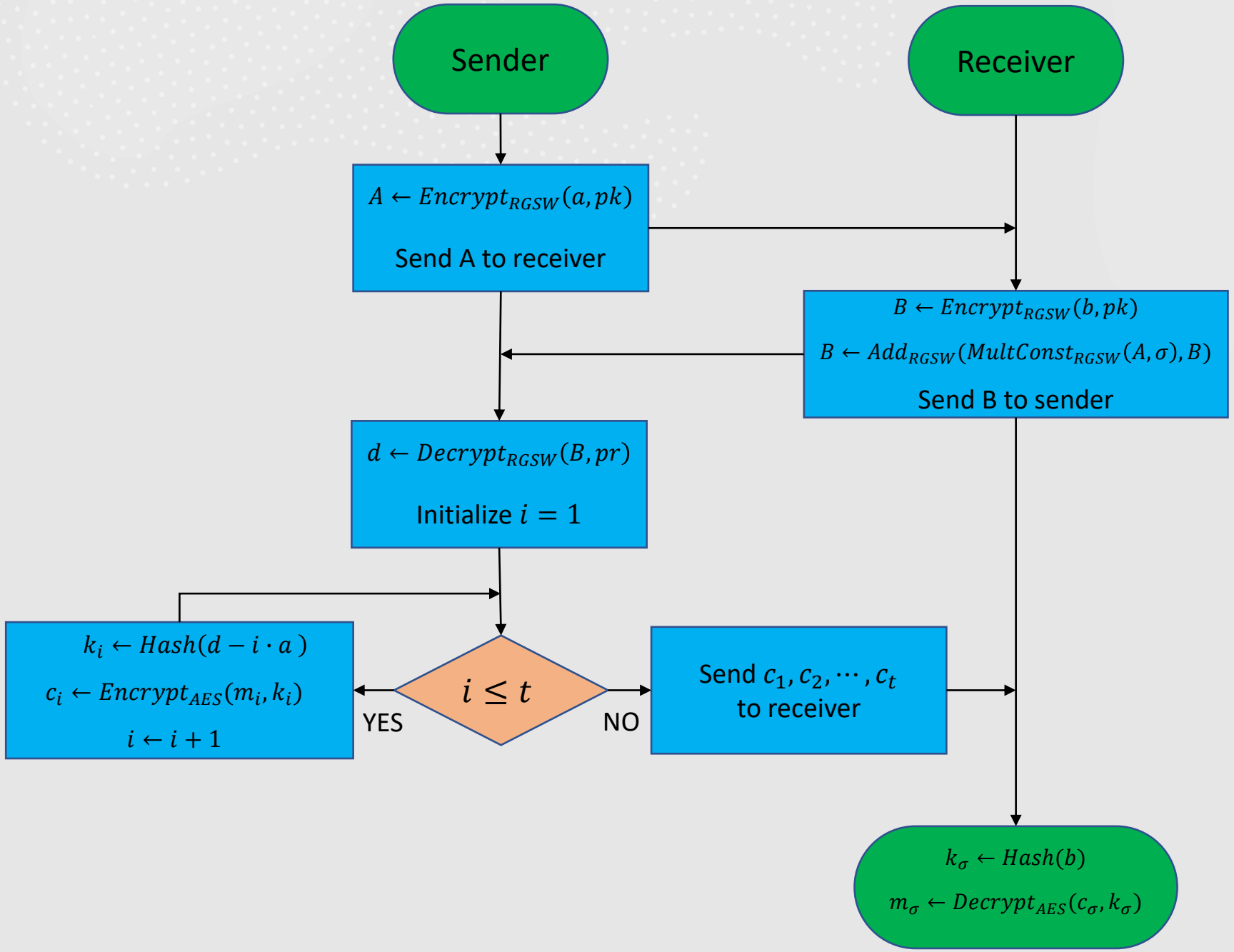
Problem Statement

- There is a need for a secure and reliable OT with the capacity to resist attacks from quantum algorithms.
- Conventional lattice-based HE schemes don't provide **full** homomorphism because of uncontrolled noise growth with each operation.
- Additionally, the existing LWE based FHE schemes suffer from huge public key and ciphertext sizes. Conversion to ring variants improve the protocol's efficiency without compromising its security.

Objectives

- To implement a lattice-based asymmetric cryptosystem with full homomorphism to enable secure computation without the knowledge of the plaintext.
- To construct a ring variant of the FHE scheme based on the RLWE assumption.
- To ensure the OT protocol is universally composable and post-quantum secure under the RLWE assumption.

Block Diagram



Algorithm

Polynomial ring $R = Z[x]/\Phi_{2n}(x)$ where $\Phi_{2n}(x)$ is a cyclotomic polynomial defined as $\Phi_{2n}(x) = x^n + 1$ with n a power of 2. Define $R_q = R/qR$, i.e., all coefficients of a polynomial in R_q are modulo q.

The coefficient modulus is $q = 2^k$ and the polynomial ring $R_q \leftarrow Z_q[x]/\Phi_{2n}(x)$ for $n = 2^{k'}$. Define $N = 2k$.

RGSW variant functions of the GSW scheme are defined for a polynomial $a \in R_q$ with coefficients $a_0, a_1, \cdots, a_{n-1} \in Z_q$

$$\overrightarrow{a'} = BitDecomp_{RGSW}(a) = \left(\sum_{i=0}^{n-1} a_{i,0} \cdot x^i, \cdots, \sum_{i=0}^{n-1} a_{i,k-1} \cdot x^i \right) \in R_q^k$$

$a_{i,j} \in Z_q$ denotes the jth LSB in a_i, therefore, $a_{i,j} \in \{0, 1\}$ for $0 \leq i < n, 0 \leq j < k$

$$BitDecomp_{RGSW}^{-1}(\overrightarrow{a'}) = \sum_{i=0}^{n-1} \left(\sum_{j=0}^{k-1} 2^j \cdot a_{i,j} \right) \cdot x^i \in R_q$$

Here, $a_{i,j} \in Z_q$ need not be only 0 or 1.

$$Flatten_{RGSW}(\vec{a}) = BitDecomp_{RGSW}\left(BitDecomp_{RGSW}^{-1}(\vec{a})\right) \in R_q^k$$
$$Powersof2_{RGSW}(a) = \left(a, 2a, \cdots, 2^{k-1}a\right) \in R_q^k$$

For a matrix $A \in R_q^{m \times n}$, all the functions above operate on each row of A.

RingGSW FHE Scheme

Setup

Let $\chi_{\mu,\sigma} \leftarrow$ Gaussian Distribution and $I_N \in R_q^{N\times N}$ is the identity matrix of order N.

Secret Key Generation: $t, \vec{v} \leftarrow SecretKeyGen_{RGSW}(q, n)$

Choose $t \in R_q$ at random, then $sk \leftarrow \vec{s} = (1, -t) \in R_q^2$. The secret key is $\vec{v} = Powersof2_{RGSW}(\vec{s}) = \left(1, 2, \cdots, 2^{k-1}, -t, -2t, \cdots, -2^{k-1}t\right) \in R_q^N$

Public Key Generation: $A \leftarrow PublicKeyGen_{RGSW}(t)$

Choose $b_1 \in R_q$ and $e \leftarrow \chi \in R_q$ is an error polynomial with small coefficients. Define $b_2 \leftarrow b_1 \cdot t + e$ then the public key is the matrix $A = [b_2 \quad b_1] \in R_q^{1\times 2}$. Observe that $A \cdot \vec{s} = \vec{e}$

Encryption: $C \leftarrow Encrypt_{RGSW}(z, A)$

Choose a plaintext $z \in R_q$ and a uniform $\vec{r} \in R_q^N$ such that the coefficients of any member polynomial are from $\{0, 1\}$. In other words, $\vec{r} \leftarrow \left(\sum_{i=0}^{n-1} y_{0,i} \cdot x^i, \cdots, \sum_{i=0}^{n-1} y_{N-1,i} \cdot x^i\right) \in R_q^N$ where $y_{i,j} \in \{0, 1\} \subset Z_q$ for $0 \le i < N, 0 \le j < k$

The ciphertext is given by $C = Flatten_{RGSW}\left(z \cdot I_N + BitDecomp_{RGSW}(\vec{r} \cdot A)\right) \in R_q^{N\times N}$

Decryption: $m \leftarrow Decrypt_{RGSW}(C, \vec{v})$

Let $z_0, z_1, \cdots, z_{k-1} \in Z_q$ be such that $\sum_{j=0}^{n-1} z_j \cdot x^j = z$ in R_q.

Consider $\vec{b} = (b_1, b_2, \cdots, b_{k-1}) = \left(z + e_1, 2z + e_2, \cdots, 2^{k-1}z + e_{k-1}\right)$.
Initialize $z \leftarrow 0 \in R_q$

for $j = 0, 1, \cdots, n-1$:

$z_j \leftarrow 0 \in Z_q$

for $i = k-1, \cdots, 1, 0$:

$$z_j \leftarrow z_j + 2^{k-1-i} \cdot \langle b_{i,j} - 2^i z_j \notin \left[-\frac{q}{4}, \frac{q}{4}\right) mod\ q \rangle\ mod\ q$$

$z \leftarrow z + z_j \cdot x^j$

Where $b_{i,j} \in Z_q$ is the coefficient of x^j in b_i and $\langle E \rangle = \begin{cases} 1, & E\ is\ true \\ 0, & E\ is\ false \end{cases}$ The $z \in R_q$ obtained is the decrypted plaintext.

Homomorphic Operations

Constant Multiplication: $C' \leftarrow MultConst_{RGSW}(C, \alpha)$

$C' = Flatten_{RGSW}(Flatten_{RGSW}(\alpha \cdot I_N) \cdot C) \in R_q^{N \times N}$ The error increases by a factor of at most N.

Addition: $C' \leftarrow Add_{RGSW}(C_1, C_2)$

$C' = Flatten_{RGSW}(C_1 + C_2) \in R_q^{N \times N}$ The correctness of this operation is immediate.

Oblivious Transfer Protocol

The sender and receiver agree upon parameters q and R_q. The sender has t messages $m_1, m_2, \cdots, m_t$ respectively. The receiver wishes to receive the message m_σ from the sender.

$$q = 2^k, R_q, N, \chi_{\mu,\sigma} \leftarrow Setup \quad t, pr \leftarrow SecretKeyGen_{RGSW}(q, n) \quad pk \leftarrow PublicKeyGen_{RGSW}(t)$$

Define $H : R_q \rightarrow Z$ to be a secure cryptographic hash function and $Encrypt_{AES}(m, k)$ and $Decrypt_{AES}(c, k)$ to be the AES encryption and decryption functions for key k, plaintext m and ciphertext c.

Sender		**Receiver**
Input: pr, pk $m_1, m_2, \cdots, m_t$		Input: $\sigma \in \{1, 2, \cdots, t\}, pk$ Output: m_σ
$a \in R_q$ $A \leftarrow Encrypt_{RGSW}(a, pk)$	$A \rightarrow$	
		$b \in R_q$ $B \leftarrow Encrypt_{RGSW}(b, pk)$ $B \leftarrow Add_{RGSW}(MultConst_{RGSW}(A, \sigma), B)$
	$\leftarrow B$	
$d \leftarrow Decrypt_{RGSW}(B, pr) \in R_q$ For $i = 1, 2, \cdots, t$: $k_i \leftarrow H(d - i \cdot a \bmod q)$ $c_i \leftarrow Encrypt_{AES}(m_i, k_i)$		
	$c_1, c_2, \cdots, c_t \rightarrow$	
		$k_\sigma \leftarrow H(b)$ $m_\sigma \leftarrow Decrypt_{AES}(c_\sigma, k_\sigma)$



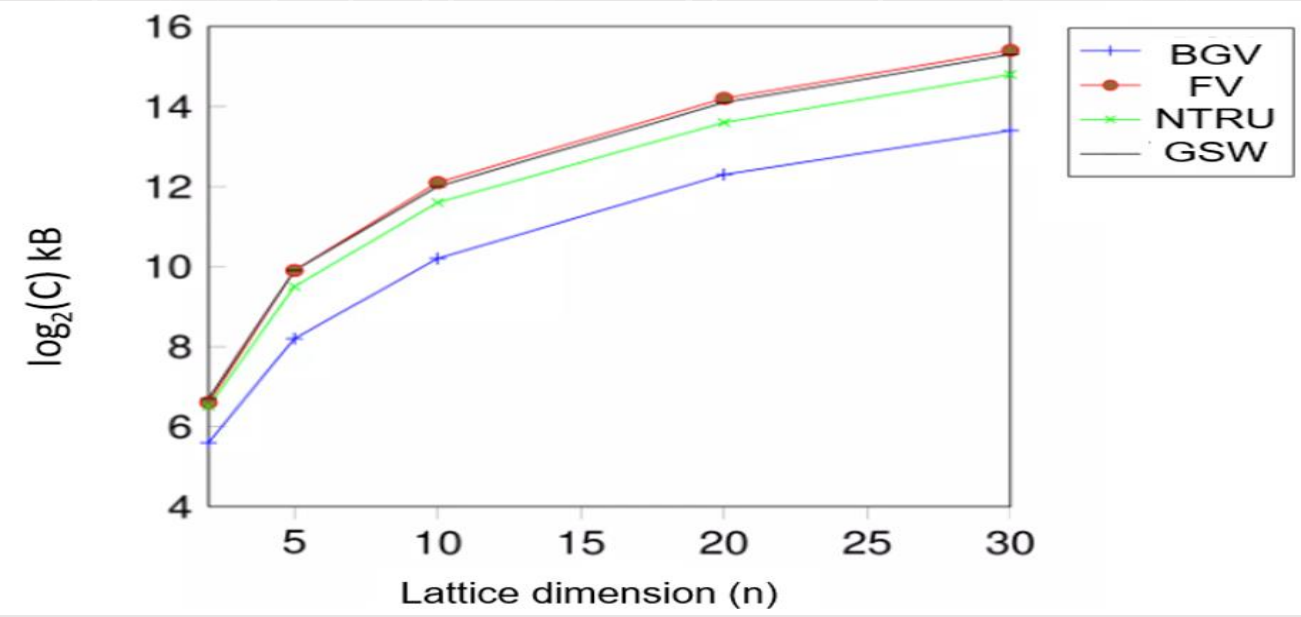
Implementation

The RGSW cryptosystem is implemented using SageMath, a free open-source mathematics software programming language.

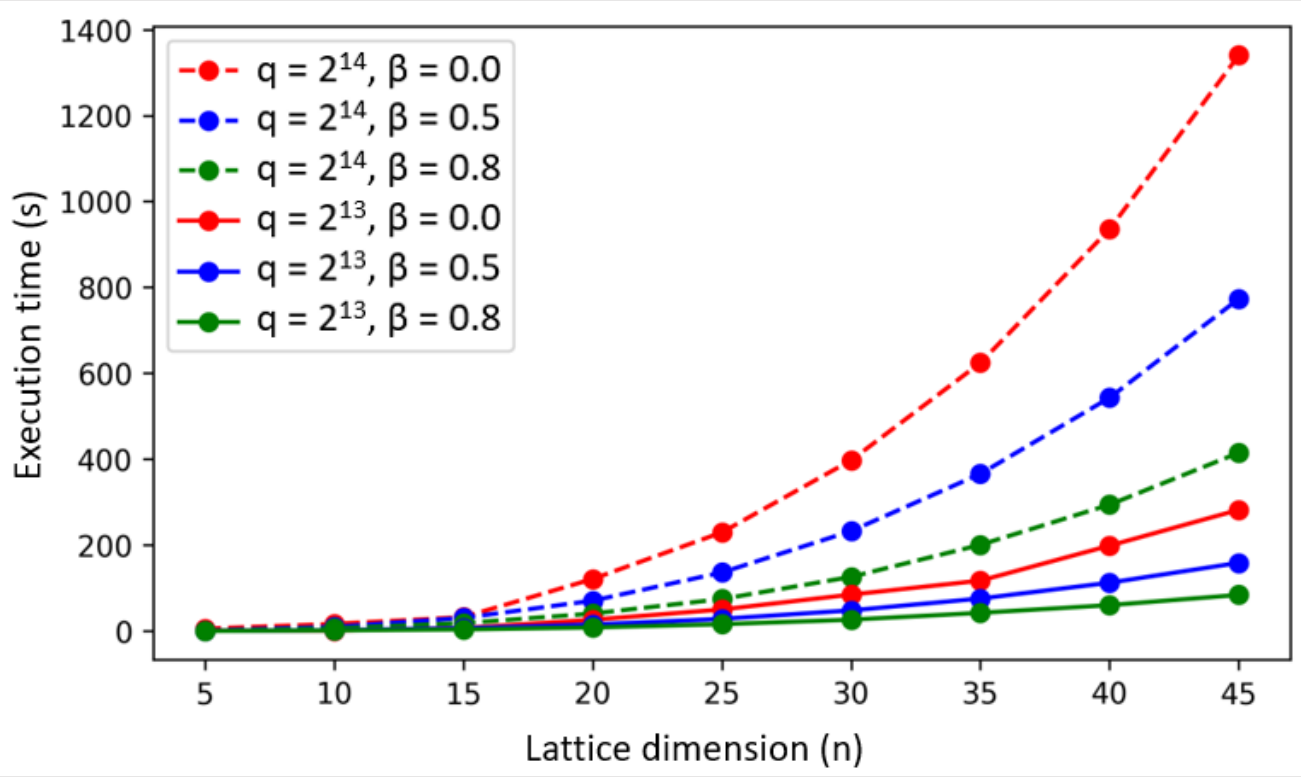
The OT protocol consists of two entities, sender and receiver. These are implemented as functions in SAGE which communicate with each other and verify the transfer as a means to test the correctness of the protocol.

As of now, we have implemented the GSW cryptosystem with full homomorphic features.

Results



Ciphertext size comparison against BGV, FV and NTRU cryptosystems



Execution time of Homomorphic operations

q – GSW modulus

β – ratio of 1s in ciphertext matrix

Timeline

Completed Modules

GSW FHE module:

1. Bootstrapping methods: `Flatten,BitDecomp,BitDecompInv,Powersof2`
2. Encryption and Decryption methods
3. Homomorphic methods: `MultConst,Add`

Remaining Modules

1. Conversion of GSW methods to RGSW.
2. OT module: `Sender, Receiver`

References

[1] Esmaeilzade, Saeid & Pakniat, Nasrollah & Eslami, Ziba. (2022). A generic construction to build simple oblivious transfer protocols from homomorphic encryption schemes. The Journal of Supercomputing. 78. 10.1007/s11227-021-03826-0.

[2] Canetti, R.. (2001). Universally Composable Security: A New Paradigm for Cryptographic Protocols. IACR Cryptology ePrint Archive. 2000. 136 - 145. 10.1109/SFCS.2001.959888.

[3] Regev, Oded. (2005). On Lattices, Learning with Errors, Random Linear Codes, and Cryptography. Journal of the ACM (JACM). 56. 84-93. 10.1145/1568318.1568324.

[4] Peikert, Chris & Pepin, Zachary. (2019). Algebraically Structured LWE, Revisited. 10.1007/978-3-030-36030-6_1.

[5] Gentry, C., Sahai, A., Waters, B. (2013). Homomorphic Encryption from Learning with Errors: Conceptually-Simpler, Asymptotically-Faster, Attribute-Based. In: Canetti, R., Garay, J.A. (eds) Advances in Cryptology – CRYPTO 2013. CRYPTO 2013. Lecture Notes in Computer Science, vol 8042. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-40041-4_5

[6] Micciancio, D., Peikert, C. (2012). Trapdoors for Lattices: Simpler, Tighter, Faster, Smaller. In: Pointcheval, D., Johansson, T. (eds) Advances in Cryptology – EUROCRYPT 2012. EUROCRYPT 2012. Lecture Notes in Computer Science, vol 7237. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-29011-4_41

[7] Chen, H., Dai, W., Kim, M., Song, Y. (2021). Efficient Homomorphic Conversion Between (Ring) LWE Ciphertexts. In: Sako, K., Tippenhauer, N.O. (eds) Applied Cryptography and Network Security. ACNS 2021. Lecture Notes in Computer Science, vol 12726. Springer, Cham. https://doi.org/10.1007/978-3-030-78372-3_18